

Dossier du projet :Création du site du Camping Le Maine Blanc



django

Nathalie Darnaudat

N° élève : F449811W

Organisme de formation : Centre Européen de Formation

Titre professionnel -

Mobile

Développeur Web et Web

Table des matières

1. Introduction	p.4
2. Remerciements	p.5
3. Compétences professionnelles mobilisées	p.6
4. Maquettage et conception de l'interface	p.8
4.1. Processus complet de maquettage	p.8
4.2. Design system et choix graphiques	p.9
4.3. Illustrations de la responsivité	p.10
4.4. Accessibilité	p.11
5. Développement front-end	p.13
5.1. Technologies utilisées	p.13
5.2. Composants dynamiques	p.13
5.3. Optimisation SEO	p.14
5.4. Sécurité front-end	p.14
5.5. Illustrations et extraits de code	p.15
6. Développement back-end	p.19
6.1. Architecture globale de l'application	p.19
6.2. Modèle de données	p.19
6.3. Composants métier	p.20
6.3.1 Création et validation des formulaires de réservation et de contact	p.20
6.3.2 Calcul automatique des prix, acomptes et suppléments	p.21

6.3.3 Gestion des disponibilités et du nombre d'occupants	p.23
6.3.4 Validation des données saisies par l'utilisateur côté serveur	p.23
6.4. Lien front/back	p.24
6.5. Sécurité back-end	p.25
6.5.1 Sécurité des données et des formulaires	p.25
6.5.2 Sécurisation des communications et de l'environnement	p.26
6.5.3 Envoi d'e-mails sécurisés	p.26
6.6. Interconnexion et transversalité	p.27
6.6.1 Interconnexion entre application	p.27
6.6.2 Internationalisation et accessibilité transversale	p.28
6.6.3 Communication front/back	p.29
6.7. Gestion de version	p.29
6.7.1 Organisation du dépôt	p.29
6.7.2 Méthodologie de travail	p.30
6.7.3 Collaboration et sauvegarde	p.31
7. Déploiement et hébergement	p.32
7.1. Mise en ligne sur Render	p.32
7.2. Sécurité et maintenance du site	p.32
7.3. Backup et mise à jour	p.33
8. Tests et validation	p.34
8.1. Scénarios de tests fonctionnels	p.34
8.2. Résultats et corrections apportées	p.36
8.3. Améliorations envisagées	p.36
9. Veille technologique et sécurité	p.37

9.1. Tests unitaires et fonctionnels (Pytest)	p.37
9.2. Scénarios de tests principaux et résultats	p.37
9.3. Difficultés rencontrées et solutions apportées	p.39
10. Analyse des difficultés et solutions	
10.1. Organisation et priorisation	p.40
10.2. Problèmes techniques front-end et back-end	p.40
10.3. Solutions mises en œuvres et apprentissages	p.41
11. Conclusion générale	
11.1. Bilan technique et compétences acquises	p.42
11.2. Clarté, pertinence et prise de recul sur le projet	p.42
11.3. Perspectives d'évolution ou améliorations futures	p.42
12. Annexes	
12.1. Arborescence du projet Django	p.44
12.2. Technologies utilisées	p.45
12.3. Exemples d'interfaces	p.45
12.4. Liens utiles	p.47
12.5. Glossaire	p.48

1. Introduction

Dans le cadre de ma fin de formation en développement web et web mobile, j'ai réalisé un projet professionnel présenté pour la validation du Titre Professionnel Développeur Web et Web Mobile (DWWM).

Ce projet, intitulé « Création du site web du Camping Le Maine Blanc », a pour objectif de moderniser un ancien site vitrine obsolète en proposant une application web complète, ergonomique, responsive, multilingue et administrable. Le site permet aux visiteurs de découvrir les hébergements, de consulter les tarifs et d'effectuer une réservation ou une demande de contact en ligne.

Le développement s'appuie sur le framework Django (Python) pour le back-end et sur les technologies HTML5, CSS3, JavaScript et Bootstrap pour le front-end. La base de données SQLite3 assure la gestion des réservations, des hébergements et des tarifs, tandis que l'administration Django offre au gérant une interface sécurisée et simple d'utilisation, pour gérer le contenu du site.

Le site est actuellement déployé sur la plateforme Render à des fins de démonstrations mais il sera ultérieurement hébergé sur un serveur professionnel afin d'assurer un niveau de performance, de sécurité et de disponibilité adapté à une situation réelle.

Le projet a été conçu selon une démarche professionnelle complète intégrant :

- la conception des interfaces et parcours utilisateurs sur Figma,
- le développement front-end et back-end dans une architecture MVC,
- l'intégration des bonnes pratiques de sécurité (CSRF, XSS, validation des formulaires, gestions des sessions),
- l'optimisation SEO (balises sémantiques, titres, URLs lisibles),
- la mise en place de tests unitaires et fonctionnels avec pytest,
- le déploiement temporaire sur Render pour la phase de démonstration.

Ce dossier présente l'ensemble des étapes de conception et de développement du site, les choix techniques réalisés, les extraits de code représentatifs, les tests menés, ainsi qu'une analyse des difficultés rencontrées et des solutions apportées. Il vise à démontrer la maîtrise des compétences techniques, méthodologiques et sécuritaires attendues pour l'obtention du titre DWWM.

2. Remerciements

Je tiens à remercier chaleureusement l'ensemble des formateurs et encadrants du Centre Européen de Formation (CEF) pour leur accompagnement, leur disponibilité et leurs précieux conseils tout au long de ma formation au titre professionnel Développeur Web et Web Mobile (DWWM).

Je remercie également mes proches pour leur soutien, leur patience et leurs encouragements constants, qui m'ont permis de mener à bien ce projet de fin de formation avec persévérance et motivation.

Enfin, j'adresse mes sincères remerciements aux responsables du Camping Le Maine Blanc, qui m'ont inspiré et permis de concevoir un site répondant à des besoins concrets et actuels dans le secteur du tourisme.

3. Compétences professionnelles mobilisées

Le projet « Création du site web du Camping Le Maine Blanc » a permis de mobiliser l'ensemble des compétences du référentiel du Titre Professionnel Développeur Web et Web Mobile (DWWM).

Bloc / Compétence	Description rapide	Mise en œuvre dans le projet
Bloc 1 - Front-end	Développer une interface utilisateur web sécurisée, responsive et accessible	HTML5, CSS3, Bootstrap 5, JavaScript, templates Django ; conception multilingue (FR/EN/ES/DE) ; accessibilité ARIA et navigation clavier ; optimisation SEO (titres, balises, URLs) ; formulaires sécurisés avec CSRF et validation serveur
Maquettage	Concevoir les interfaces et parcours utilisateur avant développement	Zoning, wireframes et prototypes réalisés sur Figma (desktop, tablette, mobile) ; suivi du design system
Bloc 2 - Back-end	Développer la logique serveur et les composants d'accès aux données	Modèles Django : Bookings (Booking, Price, SupplementPrice, MobileHome, etc.) et Core (CampingInfo, SwimmingPoolInfo, FoodInfo, LaundryInfo) ; logique métier pour calcul des prix, acomptes et disponibilités ; validation côté serveur ; base SQLite3 en développement (passage prévu à PostgreSQL en production)
Base de données & Interconnexions	Concevoir et gérer les données relationnelles et interconnectées	Tables, champs et relations modélisés dans Django ORM ; relations entre modèles ; interconnexion front/back via vues et templates
Tests & Qualité	Vérifier la fiabilité du code et la cohérence des fonctionnalités	Tests unitaires et fonctionnels avec Pytest pour les modèles, vues et formulaires ; validation de l'ensemble des règles métier

Sécurité & RGPD	Protéger les données et respecter la réglementation	Protection CSRF, XSS, injection SQL, contrôle d'accès administration ; collecte et consentement conformes RGPD ; HTTPS pour production
Déploiement & Versioning	Documenter et gérer la mise en production	Déploiement démo sur Render ; gestion des variables d'environnement ; utilisation de Git et GitHub pour versioning et suivi du code
Multilingue & Internationalisation	Offrir un contenu accessible aux utilisateurs internationaux	Utilisation de django-parler pour la gestion multilingue ; traductions DeepL automatisées pour mobil-homes et contenus dynamiques

Ce tableau offre une vue d'ensemble des compétences mises en œuvre tout au long du projet couvrant à la fois le développement front-end et back-end, la sécurisation de l'application ainsi que la gestion des données.

Les sections suivantes détailleront concrètement ces compétences à travers le processus de maquettage, la conception et le développement des interfaces, l'intégration de la base de données, les tests réalisés ainsi que la mise en œuvre des bonnes pratiques en matière de sécurité et d'expérience utilisateur.

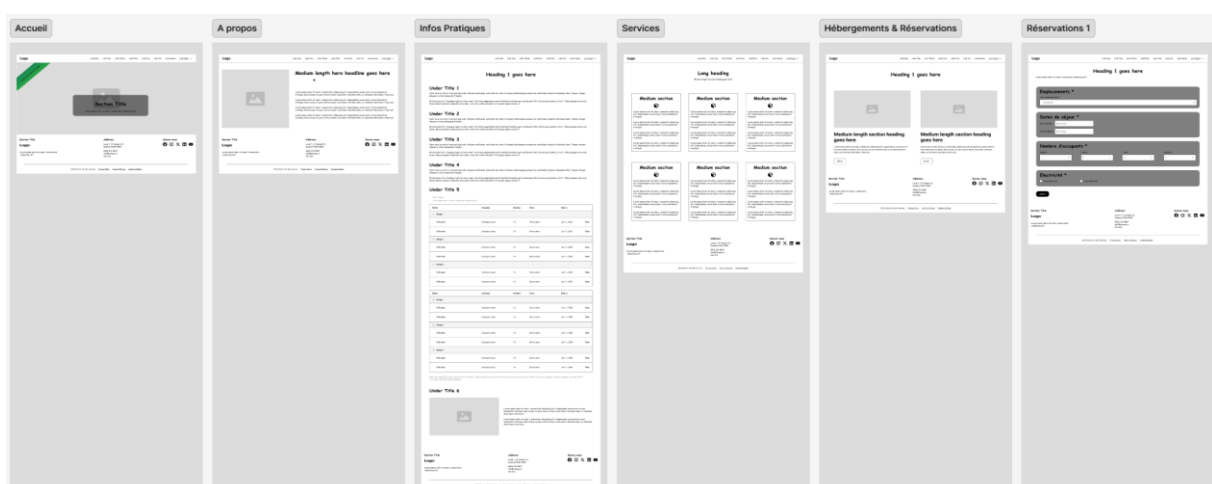
4. Maquettage et conception de l'interface

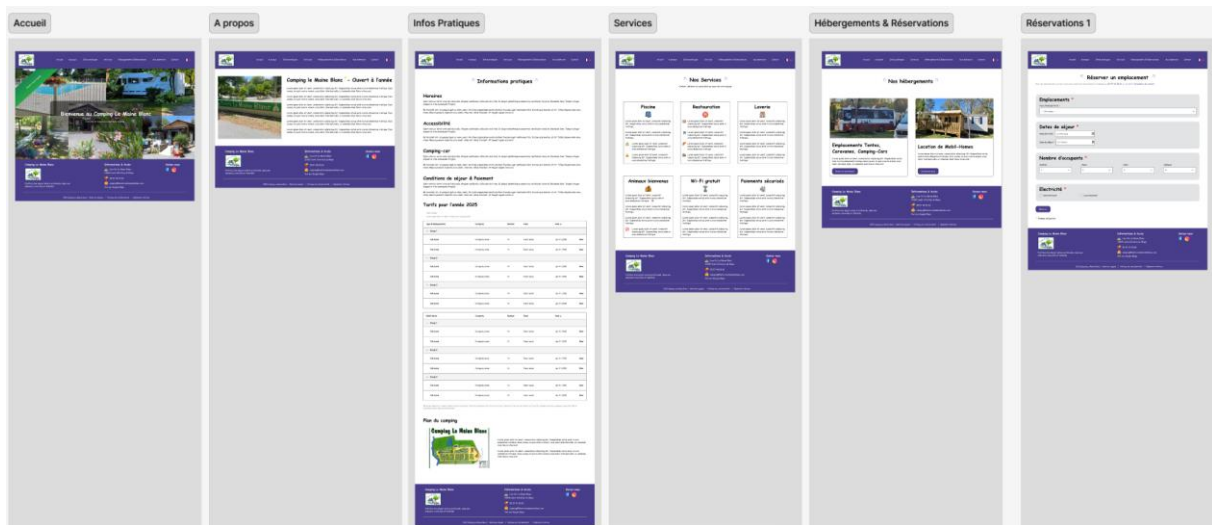
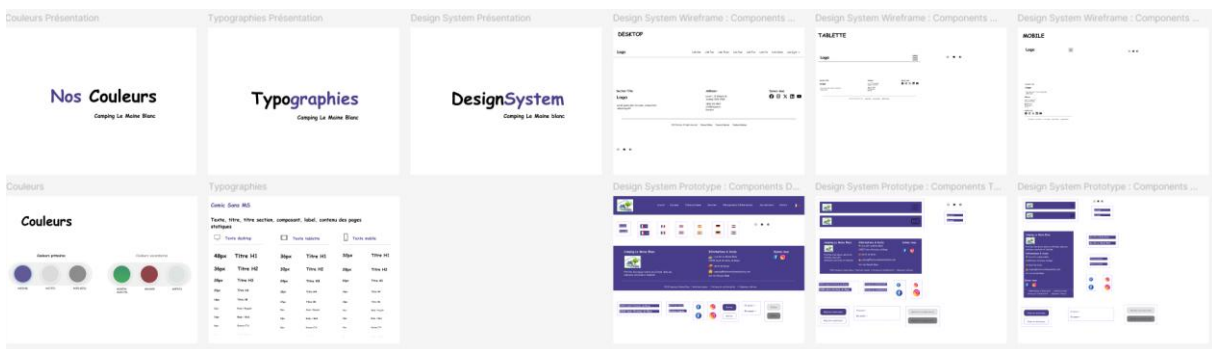
4.1. Processus complet de maquettage

Le projet a débuté par une phase de maquettage afin de structurer les pages et anticiper le parcours utilisateurs. Le processus suivi a été le suivant :

- Zoning : une première structuration des pages a été réalisée à main levée sur papier, permettant de positionner les éléments principaux (header, footer, menu, sections) et de visualiser rapidement la hiérarchie des informations.
- Wireframes : réalisation des maquettes fonctionnelles numériques sur Figma, page par page, afin de représenter les composants, la navigation et la disposition du contenu.
- Prototype interactif : création d'un prototype cliquable sur Figma, permettant de simuler la navigation entre les pages principales (Accueil, À propos, Informations pratiques, Services, Hébergements & Réservations, Aux alentours, Nous contacter).
- Design System : définition des couleurs, typographies, boutons et composants réutilisables pour garantir une cohérence graphique sur l'ensemble du site.

Wireframes :



Prototype :*Design System :*

4.2. Design system et choix graphiques

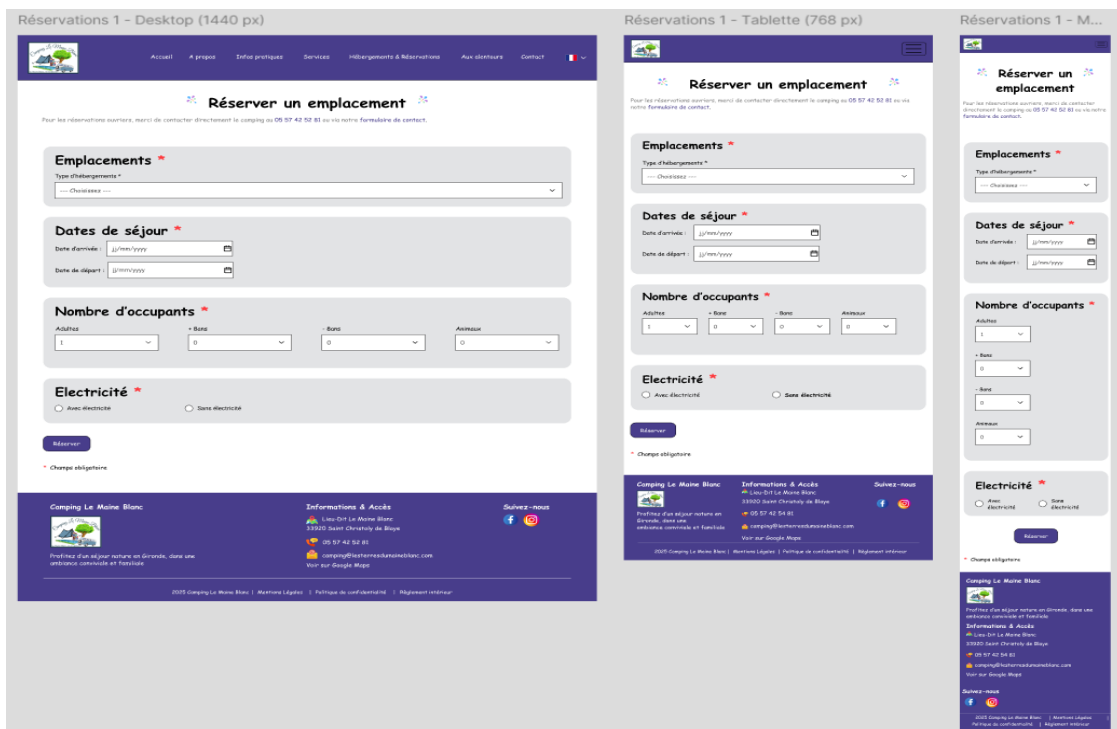
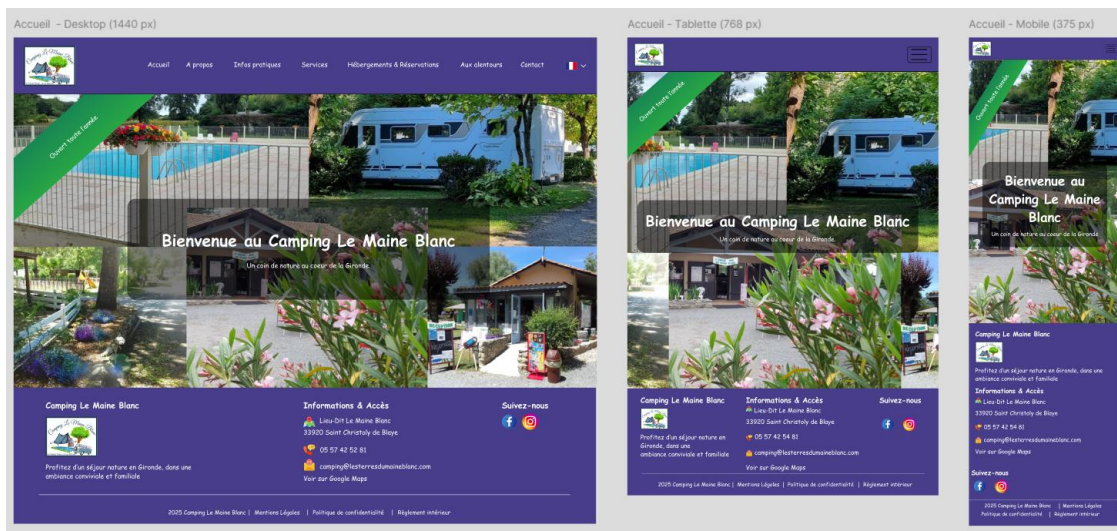
Le choix graphique a été guidé par les besoins du camping et les bonnes pratiques UX/UI :

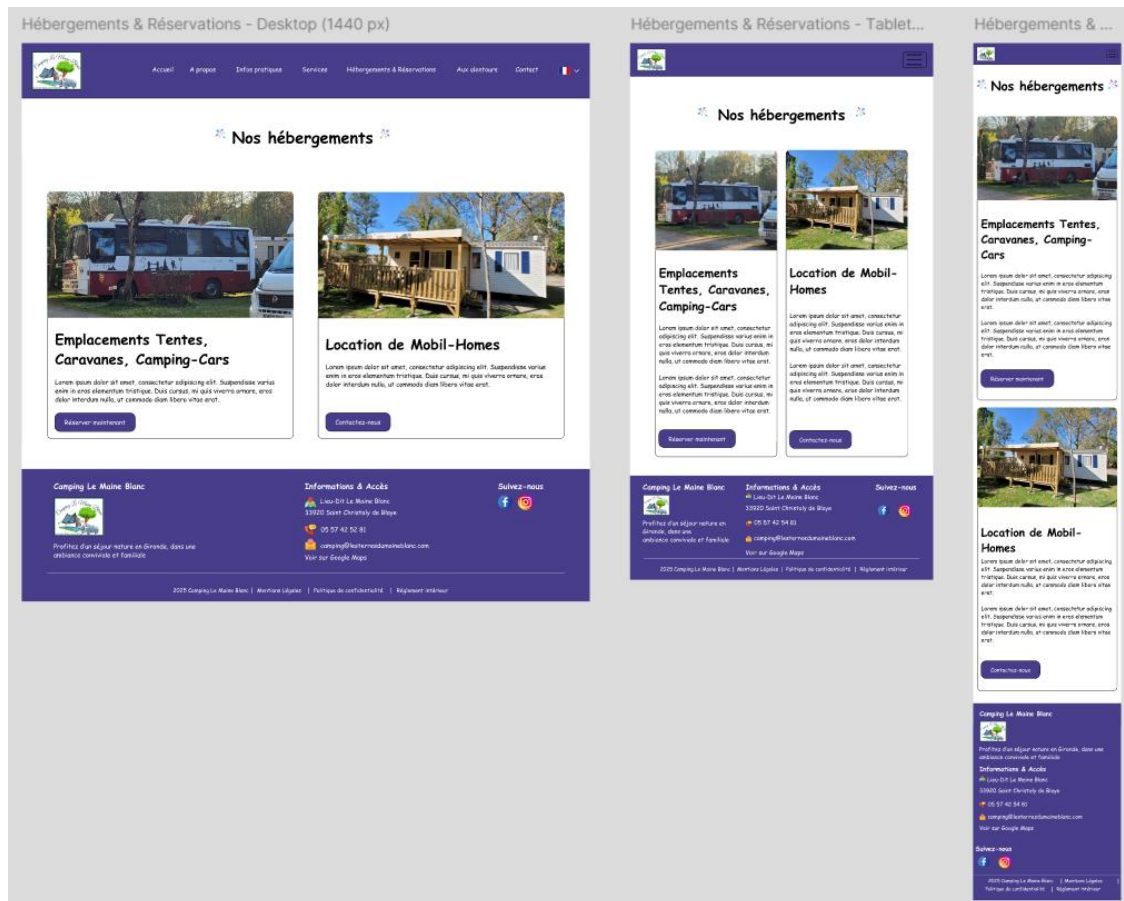
- Palette de couleurs : tons naturels rappelant l'environnement du camping, combinés à des couleurs contrastées pour les éléments interactifs (boutons, liens).
- Typographie : police sans-serif moderne pour la lisibilité et la clarté du texte sur tous les supports.
- Composants UI : boutons, formulaires, cartes d'information et tableaux de tarifs standardisés, intégrés via Bootstrap 5 pour garantir cohérence et responsivité.
- Images et icônes : utilisation d'illustrations légères et adaptées aux performances web, optimisées pour le SEO.

4.3. Illustrations de la responsivité

Le site a été conçu pour être responsive, s'adaptant aux écrans de bureau, tablette et mobile :

- Menu et navigation : menu hamburger sur mobile, affichage horizontal sur desktop.
- Grilles et sections : utilisation de la grille Bootstrap pour adapter automatiquement les colonnes et les contenus textuels et visuels.
- Formulaire et tableaux de tarifs : redimensionnement dynamique pour éviter le scroll horizontal sur petits écrans.





4.4. Accessibilité

L'accessibilité du site web a été prise en compte afin de garantir une expérience utilisateur optimale pour tous, y compris les personnes en situation de handicap. Plusieurs bonnes pratiques ont été appliquées conformément aux standards WCAG 2.1 :

- **Contraste des couleurs** : les couleurs du texte et des éléments interactifs respectent un contraste suffisant pour une lecture aisée, même pour les personnes malvoyantes
- **Navigation au clavier** : l'ensemble des menus, formulaires et liens est accessible via la navigation au clavier
- **Utilisation de balises ARIA** : des attributs ARIA (aria-label, aria-required, role) ont été ajoutés pour améliorer la compréhension des composants par les lecteurs d'écran
- **Formulaires sécurisés et lisibles** : chaque champ de formulaire est correctement étiqueté (<label> associé à l'id du champ) et les messages d'erreur sont accessibles :

```
<section class="container py-5" role="region" aria-labelledby="booking-title">
  <div class="row justify-content-center">
    <div class="col-lg-6">
      <div class="container-title text-center">...
    </div>

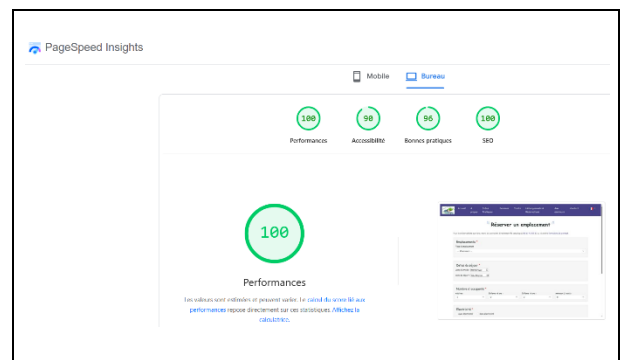
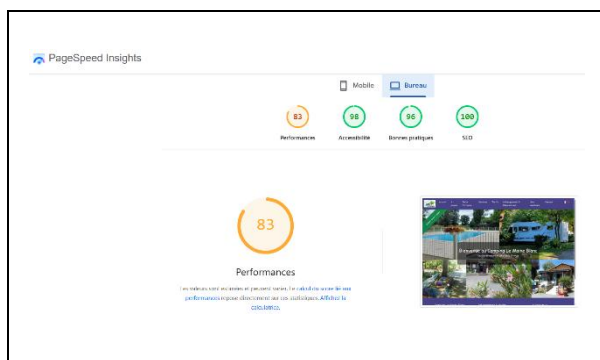
    {% if form.errors %}
    <div class="alert alert-danger" role="alert" style="border-radius: 8px; padding: 12px 16px;">...
    </div>
    {% endif %}

    {% if messages %}...
    {% endif %}

    <div class="card shadow-lg border-0 rounded-3">
      <div class="card-body">
        <form method="post" novalidate>
          {% csrf_token %}

          <div class="mb-3">
            <label for="{{ form.last_name.id_for_label }}" class="form-label">{{ form.last_name.label }}
            <span class="text-danger">*</span>
          </label>
          {{ form.last_name }}
        </div>
      </div>
    </div>
  </div>
</section>
```

- Tests automatisés d'accessibilité : les pages principales ont été analysées avec Google Lighthouse, permettant de détecter et corriger d'éventuelles erreurs d'accessibilité et d'obtenir un score global satisfaisant.



Ces mesures garantissent que le site est utilisable par le plus grand nombre et conforme aux bonnes pratiques du web accessible.

5. Développement front-end

5.1. Technologies et choix techniques

L'interface utilisateur du site Camping Le Maine Blanc a été développée avec un souci de modernité, de cohérence graphique et de performance.

Le choix de Bootstrap 5 a permis d'obtenir un design responsive et homogène sur l'ensemble des pages, tandis que l'usage de HTML5 et CSS3 a garanti une structure sémantique et conforme aux standards du web.

Le JavaScript a été utilisé pour la gestion des interactions dynamiques et des comportements conditionnels (affichage d'éléments, validation, messages utilisateurs).

Les templates Django ont permis de centraliser les éléments communs (header.html, footer.html, navbar.html) et d'éviter la redondance du code favorisant la maintenabilité et la cohérence visuelle du site.

Le site comprend plusieurs pages principales : Accueil, À propos, Services, Hébergements et réservations, Informations pratiques, Aux alentours et Nous contacter.

Chaque page a été conçue pour offrir une navigation fluide, une hiérarchie claire des informations et une mise en page responsive adaptée à tous les supports (desktop, tablette, mobile).

5.2. Composants dynamiques

Des fonctionnalités interactives ont été intégrées afin d'améliorer l'expérience utilisateur :

- Menu de navigation responsive utilisant les composants Bootstrap et un script JavaScript personnalisé.
- Formulaire dynamiques (réservation et contact) pour afficher des messages d'erreur et de confirmation en direct et désactiver des champs selon le type d'hébergement.
- Comportements interactifs (accordéons, boutons animés) pour rendre la navigation intuitive.

Ces interactions renforcent la convivialité, la clarté du parcours utilisateur et participent à la valorisation du camping.

5.3. Optimisation SEO

Pour garantir un bon référencement naturel (SEO), plusieurs optimisations ont été mises en œuvre :

- Utilisation structurée des balises sémantiques HTML5 (<header>, <nav>, <main>, <section>, <footer>),
- Rédaction optimisée des balises titres et meta descriptions avec des URLs lisibles,
- Organisation du contenu selon une hiérarchie logique des titres (<h1>, <h2>, <h3>, ...),
- Nommage cohérent des images et ajout de textes alternatifs (alt),
- Ajout d'un fichier robots.txt et d'un sitemap.xml pour faciliter l'indexation par les moteurs de recherche,
- Réduction du poids des images avant intégration pour améliorer le temps de chargement,
- Tests de performance réalisés avec Lighthouse (Google Chrome).

Ces optimisations contribuent à une meilleure visibilité du site sur les moteurs de recherche et à une expérience utilisateur fluide sur tous les appareils.

5.4. Sécurité front-end

La sécurité des échanges et des données a été intégrée dès la phase de conception. Sur la partie front-end, plusieurs mesures ont été appliquées :

- Protection CSRF sur tous les formulaires grâce à la balise {% csrf_token %} empêchant les attaques par falsification de requêtes intersites,
- Validation des formulaires côté client (JavaScript) pour réduire les erreurs de saisie et limiter les risques d'injection,
- Encodage systématique des données affichées dans les templates pour prévenir les attaques XSS,
- Gestion sécurisée des messages d'erreur afin de ne jamais exposer d'informations sensibles à l'utilisateur.

Ces mesures combinées aux protections offertes par Django côté serveur (validation, authentification, contrôle des accès) assurent une sécurité complète que nous détaillerons dans la partie suivante.

5.5. Illustrations et extraits de code

Cette section présente quelques extraits significatifs du développement front-end :

- Extrait de template base.html montrant une structure commune et l'intégration du menu responsive

```
<nav class="navbar navbar-expand-lg fixed-top" aria-label="Main navigation">
  <div class="container-fluid">
    <!-- Logo -->
    <a class="navbar-brand" href="{% url 'home' %}">
      
    </a>

    <!-- Toggle button for mobile view -->
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav"
      aria-controls="navbarNav" aria-expanded="false" aria-label="{% trans 'Basculer la navigation' %}">
      <span class="navbar-toggler-icon"></span>
    </button>

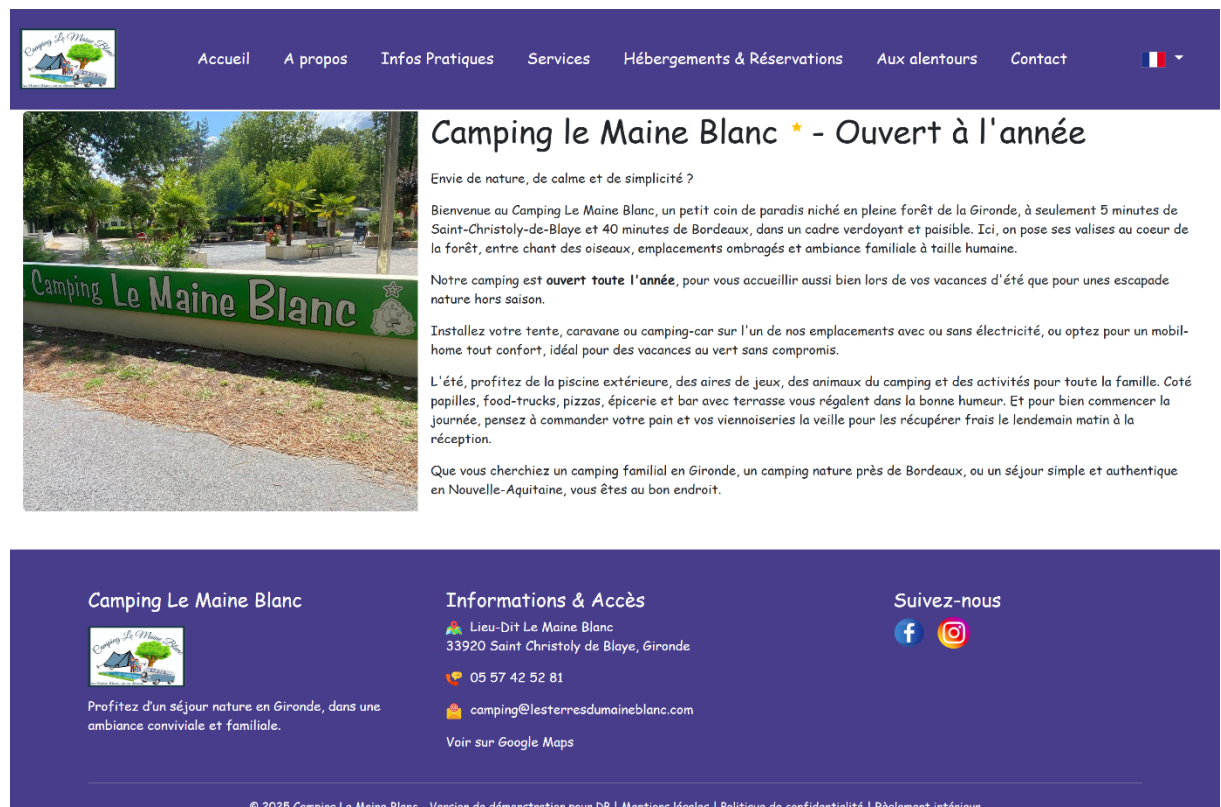
    <!-- Menu -->
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav ms-auto">...
    </ul>
    </div>
  </div>
</nav>
```

- Extrait du formulaire de contact avec gestion CSRF et messages de confirmation

```
<div class="row justify-content-center">
  <div class="col-12 col-lg-10">
    {% if form.errors %}
      <div class="alert alert-danger" role="alert" style="border-radius: 8px; padding: 12px 16px;">
        <h3 class="mb-2"> {% trans "Veuillez corriger les erreurs suivantes : " %}</h3>
        <ul class="mb-0">
          {% for field in form %}...
          {% endfor %}
          {% for error in form.non_field_errors %}...
          {% endfor %}
        </ul>
      </div>
    {% endif %}
    {% if messages %}
      {% for message in messages %}
        <div class="alert alert-success" role="alert">
          {{ message }}
        </div>
      {% endfor %}
    {% endif %}

    <form method="post" action='{% url "reservation_request" %}' novalidate>
      {% csrf_token %}
      <div class="row">...
    </div>
    <p class="small text-muted mt-3">...
    </p>
    <div class="text-center mt-4">...
    </div>
  </form>
</div>
</div>
```


- Captures d'écran du site sur les formats desktop, tablette et mobile



The screenshot shows the desktop version of the Camping Le Maine Blanc website. The header is a dark purple bar with the logo on the left and navigation links: Accueil, A propos, Infos Pratiques, Services, Hébergements & Réservations, Aux alentours, Contact, and a French flag icon. The main content area features a large photo of the campsite entrance with a green sign that reads 'Camping Le Maine Blanc'. To the right of the photo is the title 'Camping le Maine Blanc ★ - Ouvert à l'année' followed by several paragraphs of text describing the campsite's location, facilities, and opening hours. The footer is a dark purple bar with three columns: 'Camping Le Maine Blanc' with a small logo and description, 'Informations & Accès' with contact details and a Google Maps link, and 'Suivez-nous' with social media icons for Facebook and Instagram. A copyright notice is at the bottom.



Camping le Maine Blanc ★ - Ouvert à l'année

Envie de nature, de calme et de simplicité ?

Bienvenue au Camping Le Maine Blanc, un
petit coin de paradis niché en pleine



Camping le Maine Blanc ★ - Ouvert à l'année

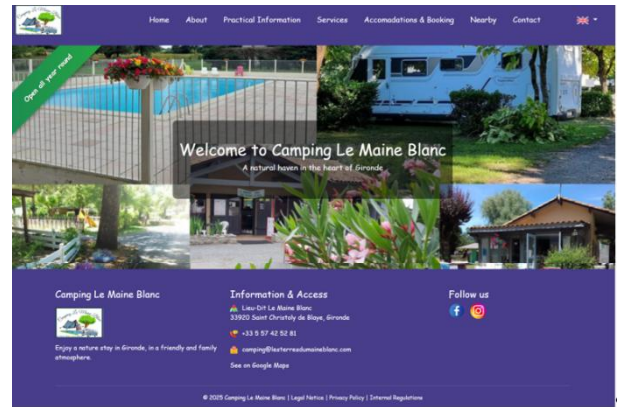
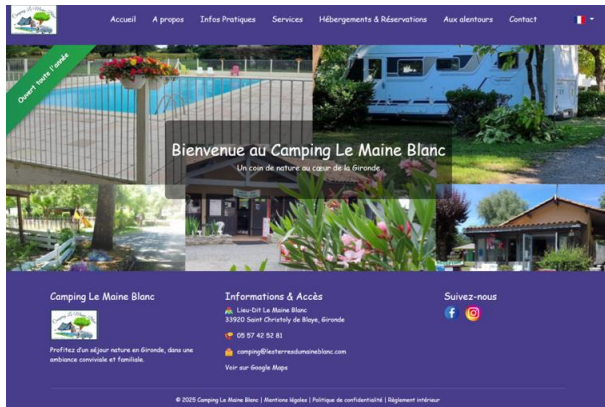
Envie de nature, de calme et de simplicité ?

Bienvenue au Camping Le Maine Blanc, un petit coin de paradis niché en pleine forêt de la Gironde, à seulement 5 minutes de Saint-Christoly-de-Blaye et 40 minutes de Bordeaux, dans un cadre verdoyant et paisible. Ici, on pose ses valises au cœur de la forêt, entre chant des oiseaux, emplacements ombragés et ambiance familiale à taille humaine.

Notre camping est **ouvert toute l'année**, pour vous accueillir aussi bien lors de vos vacances d'été que pour unes escapade nature hors saison.

Installez votre tente, caravane ou camping-car sur l'un de nos emplacements avec ou sans électricité, ou optez pour un mobil-home tout confort, idéal pour des vacances au vert sans compromis.

- Exemple d'interface multilingue



6. Développement back-end

6.1. Architecture globale de l'application

Le back-end du site Camping Le Maine Blanc repose sur le framework Django (version 5), choisi pour sa robustesse, sa sécurité native et sa structure modulaire facilitant la maintenance et l'évolution du projet.

Le site suit une architecture MVC (Modèle-Vue-Contrôleur) propre à Django :

- Modèles (Models) : définissent la structure et les relations de la base de données,
- Vues (Views) : gèrent la logique de traitement et les interactions entre l'utilisateur et les données,
- Templates : affichent les pages et les formulaires côté client.

L'application est découpée en trois modules principaux :

- core : informations générales du camping (à propos, informations pratiques, services, activités aux alentours),
- bookings : gestion des réservations et de la tarification,
- reservations : gestion des formulaires de contact et envoi de demandes en ligne.

Cette organisation favorise la clarté du code et la séparation des responsabilités.

6.2. Modèles de données

En phase de développement, la base de données SQLite3 a été utilisée pour sa simplicité d'intégration.

Le passage à PostgreSQL est prévu lors du déploiement en production afin de bénéficier d'une meilleure gestion des connexions, de la scalabilité et des contraintes d'intégrité.

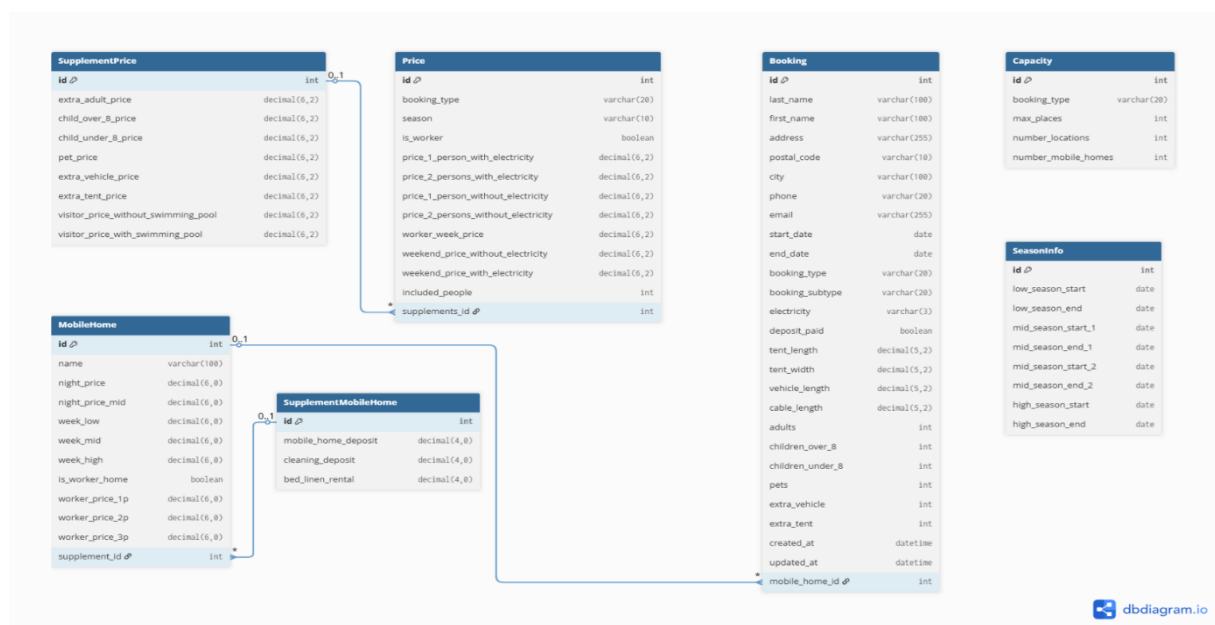
Les modèles principaux sont :

- Bookings (SupplementPrice, Price, OtherPrice, Capacity, Booking, MobilHome, SupplementMobileHome, SeasonInfo) : gèrent les tarifs, les capacités, les périodes, les types d'hébergements et les réservations en ligne.

- Core (CampingInfo, SwimmingPoolInfo, FoodInfo, LaundryInfo) : gèrent les informations générales affichées sur le site.
- Reservations : n'a pas de table propre, cette app gère uniquement les formulaires de contact et d'envoi d'e-mails sécurisés.

Les relations entre les entités suivent une logique hiérarchique simple : un hébergement peut avoir plusieurs tarifs et suppléments, et une réservation est liée à un type d'hébergement.

Un diagramme MCD a été documenté pour représenter les relations entre les tables (Booking ↔ MobilHome ↔ Supplement, etc.).



6.3. Composants métier

La logique métier repose sur des vues Django (fonctions ou classes) qui assurent la gestion des processus.

6.3.1. Création et validation des formulaires de réservation et de contact

- Déclaration des champs principaux

```
booking_type = forms.ChoiceField(
    choices=[(' ', _('Choisissez ---'))] + BOOKING_TYPE_CHOICES_FOR_FORM,
    widget=forms.Select(attrs={'class': 'form-select'}),
    label=_('Type d\'emplacement'),
    required=True
)

start_date = forms.DateField(
    widget=forms.DateInput(attrs={'type': 'date', 'min': timezone.localtime().isoformat()}),
    label=_('Date d\'arrivée'),
    required=True
)
```

Ces champs permettent de choisir le type d'emplacement et la date d'arrivée. La date minimale est fixée au jour actuel pour éviter les réservations dans le passé.

- Validation des dates

```
# Date validation
if start_date < today:
    raise forms.ValidationError(_("La date d'arrivée ne peut pas être antérieure à aujourd'hui."))
if start_date > end_date:
    raise forms.ValidationError(_("La date de départ doit être postérieure à la date d'arrivée."))
```

On s'assure que les dates sont cohérentes et que la réservation n'est pas antérieure à aujourd'hui.

- Validation conditionnelle selon le type d'emplacement

```
if booking_subtype in ["caravan", "fourgon", "van", "camping_car"]:
    if not cleaned_data.get("vehicle_length"):
        self.add_error("vehicle_length", _("Le champ 'Longueur du véhicule' est obligatoire pour les caravanes et \
" camping-cars."))

if booking_subtype in ["tent", "car_tent"]:
    if not cleaned_data.get("tent_width"):
        self.add_error("tent_width", _("Le champ 'Largeur de la tente' est obligatoire pour les tentes."))
    if not cleaned_data.get("tent_length"):
        self.add_error("tent_length", _("Le champ 'Longueur de la tente' est obligatoire pour les tentes."))
```

Certains champs deviennent obligatoires selon le type d'emplacement choisi.

- Validation de l'électricité

```
if electricity == "yes":
    cable_length = cleaned_data.get("cable_length")
    if not cable_length:
        self.add_error("cable_length", _("Le champ 'Longueur du câble' est obligatoire si l'électricité est incluse."))
```

Si l'utilisateur choisit l'option électricité, la longueur du câble devient obligatoire.

6.3.2. Calcul automatique des prix, acomptes et suppléments

- Détermination du nombre de nuits et du type d'emplacement

```
nights = max((self.end_date - self.start_date).days, 1)

subtype_to_type_map = {
    'tent': 'tent',
    'car_tent': 'tent',
    'caravan': 'caravan',
    'fourgon': 'caravan',
    'van': 'caravan',
    'camping_car': 'camping_car',
}

booking_type_for_price = subtype_to_type_map.get(self.booking_subtype, self.booking_type)
```

Le nombre de nuits est toujours au minimum d'une nuit.

Une correspondance (subtype_to_type_map) permet d'associer chaque sous-type de réservation général afin d'appliquer la bonne grille tarifaire.

- Récupération du tarif selon la saison et le type

```
season = self.get_season()
try:
    price = Price.objects.get(
        booking_type=booking_type_for_price,
        is_worker=False,
        season=season
    )
```

Le prix de base est récupéré dans le modèle Price, selon la saison et le type d'emplacement.

Le système est ainsi flexible et permet d'adapter automatiquement les tarifs selon la période (haute, moyenne ou basse).

- Application du tarif de base selon le type et l'électricité

```
if booking_type_for_price == 'camping_car':
    base_price = price.price_2_persons_with_electricity if electricity_yes else price.price_2_persons_without_electricity
    included_people = 2
else:
    included_people = 2 if self.adults >= 2 else 1
    if self.adults >= 2:
        base_price = price.price_2_persons_with_electricity if electricity_yes else price.price_2_persons_without_electricity
    else:
        base_price = price.price_1_person_with_electricity if electricity_yes else price.price_1_person_without_electricity
```

Le prix de base dépend du nombre d'adultes et de la présence d'électricité.

Cette logique garantit que le calcul soit juste et adapté à la situation du client.

- Ajout des suppléments (adultes supplémentaires, enfants, animaux)

```
if supplement:
    total += extra_adults * (supplement.extra_adult_price or 0) * nights
    total += self.children_over_8 * (supplement.child_over_8_price or 0) * nights
    total += self.children_under_8 * (supplement.child_under_8_price or 0) * nights
    total += self.pets * (supplement.pet_price or 0) * nights
    total += self.extra_vehicle * (supplement.extra_vehicle_price or 0) * nights
    total += self.extra_tent * (supplement.extra_tent_price or 0) * nights
```

Chaque option ou supplément est ajouté au prix total selon le tarif journalier correspondant.

Le code reste modulaire : il est facile d'ajouter de nouveaux suppléments à l'avenir.

- Calcul de l'acompte

```
def calculate_deposit(self):
    """Calculate 15% deposit of total price, rounded to 2 decimals."""
    total_price = self.calculate_total_price()
    return round(total_price * Decimal('0.15'), 2)
```

Une fois le prix total calculé, un acompte de 15% est appliqué pour valider la réservation.

Ce pourcentage est clairement défini dans le code, ce qui permet une modification simple si la politique du camping évolue.

6.3.3. Gestion des disponibilités et du nombre d'occupants

- Validation du nombre total de personnes

```
total_people = adults + children_over_8 + children_under_8

if total_people > 6:
    raise forms.ValidationError(
        _("Le nombre maximum de personnes (adultes et enfants) ne peut pas dépasser 6."
          " Pour toute demande particulière, merci de contacter directement le camping.")
    )
```

On limite le nombre total de personnes par emplacement pour respecter les règles du camping.

- Vérification de la disponibilité

```
def check_capacity(self):
    MAIN_TYPE_MAP = {
        'tent': 'tent',
        'car_tent': 'tent',
        'caravan': 'caravan',
        'fourgon': 'caravan',
        'van': 'caravan',
        'camping_car': 'camping_car',
    }
    if self.start_date is None or self.end_date is None:
        raise ValidationError(_("Les dates de réservations sont requises pour vérifier la disponibilité."))
    main_type = MAIN_TYPE_MAP.get(self.booking_type, self.booking_type)
    try:
        capacity = Capacity.objects.get(booking_type=main_type).max_places
    except Capacity.DoesNotExist:
        raise ValidationError(_("La capacité pour %(type)s n'est pas définie.") % {'type': main_type})
    overlapping = Booking.objects.filter(
        booking_type__in=[key for key, value in MAIN_TYPE_MAP.items() if value == main_type],
        start_date__lt=self.end_date,
        end_date__gt=self.start_date
    ).exclude(pk=self.pk)
    if overlapping.count() >= capacity:
        raise ValidationError(
            _("Plus de places disponibles pour ces dates. "
              "Veuillez choisir d'autres dates ou contacter le camping.")
        )
```

Pour toute création de réservation, la fonction vérifie la disponibilité suivant le type d'emplacement choisi.

6.3.4. Validation des données saisies par l'utilisateur côté serveur

- Vérification des données et intégrité de la session

```
booking_data = request.session.get('booking_data')

if not booking_data:
    messages.error(request, _("Aucune donnée de réservation trouvée. Veuillez recommencer le processus de réservation. "))
    return redirect('booking_form')

required_fields = ['first_name', 'last_name', 'address', 'postal_code', 'city', 'email', 'phone']
if not all(field in booking_data for field in required_fields):
    messages.error(request, _("Les informations de contact sont incomplètes. Veuillez compléter vos coordonnées. "))
    return redirect('booking_details')
```

Avant toute création de réservation, la fonction vérifie que les données nécessaires sont présentes dans la session.

Cela empêche qu'un utilisateur contourne les étapes précédents ou envoie une requête incomplète directement.

- Conversion et nettoyage des données avant enregistrement

```
booking_data['start_date'] = date.fromisoformat(booking_data['start_date'])
booking_data['end_date'] = date.fromisoformat(booking_data['end_date'])
booking_data = {k: v for k, v in booking_data.items() if k in model_fields}
booking = Booking(**booking_data)
```

Les dates sont converties au bon format et seules les clés correspondant aux champs du modèle Booking sont conservées.

Cette étape protège contre l'injection de données non prévues et garantit la cohérence du modèle.

- Nettoyage de la session et message utilisateur

```
if 'booking_data' in request.session:
    del request.session['booking_data']

messages.success(
    request,
    _("Merci ! Votre réservation a été confirmée. Un email de confirmation vous a été envoyé." if not is_render else
      "Merci ! Votre réservation a été confirmée. (Simulation d'envoi d'email sur Render.)")
)
```

Après la confirmation, toutes les données sont supprimées de la session afin d'éviter toute duplication si l'utilisateur recharge la page.

L'affichage d'un message de confirmation renforce l'expérience utilisateur.

Le back-end interagit directement avec les templates via le moteur de rendu Django, garantissant une séparation claire entre le code Python et le HTML.

6.4. Lien front/back

Les vues Django fournissent les données aux templates pour l'affichage côté front. Les formulaires sont validés côté serveur et les messages d'erreur et de confirmation sont rendus directement dans les templates.

```

<div class="row justify-content-center">
  <div class="col-12 col-lg-10">
    {% if form.errors %}
      <div class="alert alert-danger" role="alert" style="border-radius: 8px; padding: 12px 16px;">
        <h3 class="mb-2"> {% trans "Veuillez corriger les erreurs suivantes : " %}
        </h3>
        <ul class="mb-0">
          {% for field in form %}...
          {% endfor %}
          {% for error in form.non_field_errors %}...
          {% endfor %}
        </ul>
      </div>
    {% endif %}
    {% if messages %}
      {% for message in messages %}
        <div class="alert alert-success" role="alert">
          {{ message }}
        </div>
      {% endfor %}
    {% endif %}
    <form method="post" action='{% url "reservation_request" %}' novalidate>
      {% csrf_token %}
      <div class="row">...
    </div>
    <p class="small text-muted mt-3">...
    </p>
    <div class="text-center mt-4">...
    </div>
  </form>
</div>
</div>

```

6.5. Sécurité back-end

Django offre un haut niveau de sécurité par défaut, renforcé par des pratiques spécifiques appliquées dans ce projet :

6.5.1. Sécurité des données et des formulaires

- Protection CSRF (Cross-Site Request Forgery)

Tous les formulaires du site incluent le jeton CSRF fourni par Django afin d'empêcher l'envoi de requêtes malveillantes.

```

<form method="post" action='{% url "reservation_request" %}' novalidate>
  {% csrf_token %}
  <div class="row">...
</div>

  <p class="small text-muted mt-3">...
  </p>

  <div class="text-center mt-4">...
  </div>
</form>

```

Ce mécanisme vérifie que les requêtes POST proviennent bien du site légitime et non d'une source externe.

- Validation et nettoyage des données

Les formulaires, notamment le BookingForm, effectuent une validation stricte des champs (types, formats, plages de dates).

```
if start_date < today:
    raise forms.ValidationError(_("La date d'arrivée ne peut pas être antérieure à aujourd'hui."))
```

Cette étape empêche les saisies incohérentes ou malveillantes d'être enregistrées dans la base de données.

- Filtrage des champs autorisés

Avant toute sauvegarde, les vues vérifient que seules les données attendues sont traitées.

```
model_fields = [f.name for f in Booking._meta.get_fields()]
booking_data = {k: v for k, v in booking_data.items() if k in model_fields}
```

Cela protège contre les attaques de type mass assignment (injection de champs non autorisés).

6.5.2. Sécurisation des communications et de l'environnement

- HTTPS

Le site est configuré pour fonctionner sous HTTPS avec certificat SSL, assurant le chiffrement des échanges entre le serveur et les utilisateurs (formulaires, e-mails, cookies).

- Gestion sécurisée des variables sensibles

Les paramètres sensibles (mots de passe, clés secrètes, identifiants SMTP, ...) sont stockés dans un fichier .env non versionné :

```
EMAIL_HOST_PASSWORD = config('EMAIL_HOST_PASSWORD', default='')
STRIPE_SECRET_KEY = config('STRIPE_SECRET_KEY', default='')
STRIPE_PUBLIC_KEY = config('STRIPE_PUBLIC_KEY', default='')
```

Cela évite toute fuite accidentelle sur un dépôt public.

Protection des sessions

Les données temporaires de réservation (booking_data) sont stockées dans la session Django, isolée et sécurisée par des cookies signés.

Elles sont supprimées automatiquement après confirmation.

```
if 'booking_data' in request.session:
    del request.session['booking_data']
```

6.5.3. Envoi d'e-mails sécurisés

Les messages envoyés aux clients et à l'administrateur utilisent le module EmailMessage de django.

Chaque envoi est encapsulé dans un bloc try/except pour intercepter les erreurs.

```

try:
    with translation.override('fr'):
        extra_info_1 = ""
        extra_info_2 = ""

        if booking.is_tent:
            extra_info_1 = _("Dimensions tente : {length} m x {width} m").format(...)
        elif booking.is_vehicle: ...
        if booking.electricity == 'yes':
            extra_info_2 = _("Longueur câble : {length} m").format(...)

        admin_subject = _("Nouvelle réservation de {booking.first_name} {booking.last_name}").format(booking=booking)
        admin_message_final = render_to_string('emails/admin_booking.html', { ...

        if is_render: ...
        else:
            email_admin = EmailMessage(...)
            email_admin.content_subtype = "html"
            email_admin.send(fail_silently=False)

# Email to client in selected language
with translation.override(request.LANGUAGE_CODE): ...

except Exception as e:
    print("⚠ Erreur lors de l'envoi des emails :", e)
    messages.warning(request, _("Votre réservation est enregistrée, mais une erreur est survenue lors de l'envoi des emails. " \
    "Veuillez contacter l'administrateur."))

```

Les e-mails sont également rendus à partir de templates HTML sécurisés (`render_to_string`), évitant toute injection de contenu dynamique non contrôlé.

6.6. Interconnexion et transversalité

L'architecture Django du projet Camping Le Maine Blanc repose sur une structure modulaire où chaque application (`bookings`, `core`, `reservations`) conserve son indépendance tout en communiquant avec les autres.

Cette transversalité garantit une évolution fluide du code sans duplication et une meilleure maintenabilité à long terme.

6.6.1. Interconnexion entre applications

- `bookings` <-> `core`

Les données tarifaires et saisonnières issues de `booking` (tarifs, suppléments, capacités, mobil-homes) sont exploitées dans les vues `core`, notamment `infos_view`, pour afficher dynamiquement les informations générales du camping afin de garantir la cohérence entre les informations visibles sur le site et les règles métiers définies dans la logique de réservation, sans duplication de données.

- `bookings` <-> `reservations`

Après une réservation ou une demande de contact, les données saisies transitent via les vues de `reservations` pour l'envoi d'e-mails automatiques (confirmation client et notification administrateur).

- `core` <-> templates

Les données affichées sur les pages publiques (tarifs, services, périodes d'ouverture) sont dynamiquement récupérées depuis le modèle core et injectées dans les templates.

Cette interconnexion repose sur des imports ciblés et une gestion rigoureuse des dépendances afin d'éviter des boucles circulaires

```
from bookings.models import Price, SupplementPrice, SeasonInfo, Capacity, MobileHome, SupplementMobileHome, OtherPrice
def infos_view(request):
    # --- Retrieve all standard and worker prices ---
    prices = Price.objects.all()
    grouped_prices = {}
    normal_prices = prices.filter(is_worker=False)
    for price in normal_prices:
        # --- Worker prices ---
        worker_prices = prices.filter(is_worker=True)
    # --- Supplements prices ---
    supplements_obj = SupplementPrice.objects.first()
    supplements = []
    visitor_prices = []
    if supplements_obj:
        # --- Camping general information ---
        camping_info = CampingInfo.objects.first()
        other_prices = OtherPrice.objects.first()
        season_info = SeasonInfo.objects.first()
        capacity_info = Capacity.objects.first()
        # --- Language-specific handling ---
        lang = get_language()
    if camping_info:
        mobilhomes = MobileHome.objects.all()
    for home in mobilhomes:
        mobilhome_supplements = SupplementMobileHome.objects.first()
    return render(request, 'core/infos.html', {
        "grouped_prices": grouped_prices,
        "worker_prices": worker_prices,
        "supplements": supplements,
        "visitor_prices": visitor_prices,
        "mobilhomes": mobilhomes,
        "mobilhome_supplements": mobilhome_supplements,
        "camping_info": camping_info,
        "season_info": season_info,
        "capacity_info": capacity_info,
        "other_prices": other_prices,
    })
```

6.6.2. Internationalisation et accessibilité transversale

L'ensemble du projet est multilingue (français, anglais, espagnol, allemand, néerlandais).

Grâce à Django i18n :

- les textes statiques sont gérés avec {% trans %} dans les templates,
- les fichiers .po et .mo sont regroupés dans le dossier locale/ pour chaque langue,
- le sélecteur de langue agit sur toutes les pages via le middleware LocaleMiddleware.

Ce système assure une expérience cohérente sur l'ensemble du site et facilite la maintenance des traductions.

6.6.3. Communication front/back

Les vues Django servent de pont entre le front-end et le back-end :

- les données des formulaires front (HTML, Bootstrap) sont envoyées en POST, validées côté serveur puis affichées dans les templates via le moteur de rendu Django,
- l'interface reste fluide et responsive, sans du JavaScript lourd car les transitions sont gérées via les messages de succès ou d'erreur renvoyés par les vues.

Cette approche garantit simplicité, sécurité et performance tout en maintenant une séparation claire entre logique serveur et interface utilisateur.

6.7. Gestion de version

Le projet est intégralement versionné avec Git, outil essentiel à la traçabilité et à la collaboration sur le code.

Cette gestion permet de suivre chaque évolution du projet et de revenir facilement à une version stable si nécessaire.

6.7.1. Organisation du dépôt

Le dépôt Git est structuré de manière à refléter la hiérarchie Django :

```
maineblanc_project/  
|  
|--- bookings          # Application de réservation en ligne  
|   |--- templates     # Fichiers HTML  
|   |--- tests          # Tests models, views et forms  
|   |--- models.py      # Modèles de données  
|   |--- views.py       # Logique de traitement  
|   |--- urls.py        # Routage  
|   |--- forms.py       # Gestion des formulaires  
|--- core               # Application principale  
|--- docs               # Gestion de la documentation  
|--- locale             # Gestion des traductions  
|--- maineblanc_project # Fichiers de configuration Django  
|--- reservations       # Application de contact  
|--- static             # Feuilles CSS, scripts JS, images  
|--- .env               # Variables de données
```

--- .gitignore	# Données non envoyées sur GitHub
---db.sqlite3	# Base de données locales
---manage.py	# Point d'entrée principal du projet
--- pytest.ini	# Initialisation des fichiers tests
--- README.md	# Fichier de documentation
---requirements.txt	# Bibliothèques utilisées

Chaque module est indépendant et documenté par un README.md interne pour faciliter la reprise du code.

6.7.2. Méthodologie de travail

Une organisation stricte des branches a été mise en place pour garantir la stabilité du code et faciliter les évolutions :

- main : contient la version stable, testée et prête au déploiement.
- dev : sert de zone d'intégration pour les nouvelles fonctionnalités avant validation.
- feature/... : chaque nouvelle fonctionnalité ou correction est développée dans une branche dédiée, par exemple :
 - feature/booking (ajout de l'app de réservation)
 - feature/translate_payment (mise en place du système de paiement)

Le processus de travail sur GitHub s'organise comme suit :

- création d'une issue sur GitHub pour décrire la fonctionnalité à ajouter ou le bug à corriger,
- création d'une branche associée :
 - git branch feature/nom-fonctionnalite
 - git checkout feature/nom-fonctionnalite,
- développement et commit avec un message clair,
- push de la branche sur GitHub et ouverture d'une pull request (main ← feature/...) pour revue et validation,
- une fois validée, la branche est fusionnée (merge) dans main, puis supprimée,
- l'issue correspondante est ensuite fermée sur GitHub.

Chaque modification majeure est validée par une vérification manuelle sur un environnement local avant fusion dans le main et au besoin un test unitaire via Pytest.

6.7.3. Collaboration et sauvegarde

Le projet est hébergé sur *GitHub* avec :

- un historique complet des commits,
- un suivi des issues (bugs, améliorations),
- une sauvegarde automatique du code sur le cloud.

Ce mode de gestion garantit une sécurité du code source, une traçabilité des évolutions et une capacité de restauration rapide en cas d'erreur.

7. Déploiement et hébergement

7.1. Mise en ligne sur Render

Le site a été déployé sur la plateforme Render à titre d'environnement de démonstration.

Cette solution permet de présenter le projet en ligne de manière fonctionnelle, tout en conservant une configuration proche de la production.

Les étapes principales du déploiement sont :

- connexion du dépôt GitHub à Render pour bénéficier du déploiement automatique après chaque push sur la branche main,
- configuration du service Web via la commande :
`gunicorn maineb blanc_project.wsgi:application`,
- ajout des variables d'environnement essentielles (clé secrète Django, DEBUG, ALLOWED_HOSTS),
- utilisation de la base SQLite3 pour cette version démo, adaptée à un usage ponctuel et sans forte charge (le passage à PostgreSQL est prévu lors du déploiement final sur l'hébergeur définitif, pour une meilleure performance et une gestion multi-utilisateurs plus robuste),
- vérification des principales fonctionnalités en ligne (navigation, formulaires, multilingue, sécurité, redirections).

Ce déploiement temporaire sur Render permet de tester et présenter le site dans des conditions réelles avant sa mise en production finale.

7.2. Sécurité et maintenance du site

Même dans cet environnement de démonstration, plusieurs mesures ont été intégrées pour garantir la sécurité :

- HTTPS activé automatiquement par Render,
- mode DEBUG désactivé en ligne,
- configuration stricte des ALLOWED_HOSTS,
- protection CSRF et validation côté serveur pour les formulaires,
- logs Render utilisés pour surveiller les erreurs (HTTP 404, 500),
- contrôle des dépendances et mise à jour régulière via `pip install --upgrade`.

Ces précautions assurent un niveau de sécurité suffisant pour un site en démonstration, tout en préparant une transition fluide vers un hébergeur professionnel.

7.3. Backup et mise à jour

Pour cet environnement temporaire :

- des copies locales de la base SQLite3 sont régulièrement conservées (sauvegarde du fichier db.sqlite3),
- les fichiers statiques et médias sont gérés localement puis synchronisés avec Render lors du déploiement,
- avant toute mise à jour de code, un test local complet est effectué pour éviter toute régression.

Lors du déploiement final, le processus évoluera vers :

- une sauvegarde automatisée via PostgreSQL (pg_dump),
- une procédure de migration planifiée (python manage.py migrate),
- un suivi des logs serveur et de la disponibilité du site via un tableau de bord hébergeur.

Ce déploiement sur Render constitue une étape intermédiaire essentielle. Il permet de valider le bon fonctionnement, la sécurité et la stabilité du site dans un environnement en ligne avant sa mise en production finale sur un hébergeur professionnel avec une base de données PostgreSQL et des sauvegardes automatisées.

8. Tests et validation

8.1. Scénarios de tests fonctionnels

Les tests fonctionnels ont pour objectif de valider le bon fonctionnement global du site en conditions réelles, depuis la page d'accueil jusqu'à la confirmation de réservation.

Ils permettent de garantir la cohérence du processus utilisateur, la fiabilité du calcul des prix, la validation des formulaires et la suppression sécurisée des données temporaires.

Les objectifs du jeu d'essai sont :

- vérifier la cohérence du processus de réservation complet,
- tester la fiabilité du calcul des prix et des acomptes,
- confirmer la sécurité et la suppression des données temporaires après réservation,
- valider l'affichage responsive sur différents supports,
- s'assurer de la bonne gestion multilingue du site.

Les tests ont été effectués manuellement via le navigateur sur le serveur local (localhost:8000) et automatiquement avec Pytest pour les modèles, formulaires et vues.

N°	Fonctionnalité testée	Description du test	Résultat attendu	Résultat obtenu
1	Accès au site	Saisir l'URL d'accueil	La page d'accueil s'affiche avec le menu et le logo	Conforme
2	Navigation multilingue	Changer la langue (FR/EN/ES/DE/NL)	Le contenu et les menus s'affichent dans la langue choisie	Conforme

3	Formulaire de réservation	Remplir et valider le formulaire avec des dates valides	Passage à l'étape suivante avec récapitulatif de la réservation	Conforme
4	Validation des champs obligatoires	Laisser un champ vide (ex : e-mail)	Message d'erreur « Ce champ est obligatoire »	Conforme
5	Vérification des dates	Entrer une date de fin antérieure à la date d'arrivée	Message d'erreur bloquant la soumission	Conforme
6	Calcul du prix	Réservation de 2 adultes, 1 enfant + électricité	Le prix total correspond au tarif du modèle Price et Supplement	Conforme
7	Calcul de l'acompte	Vérifier le calcul automatique de 15 % du total	L'acompte affiché = Total $\times$ 0,15	Conforme
8	Enregistrement en base	Soumettre la réservation complète	Les données sont enregistrées en base (Booking)	Conforme
9	Envoi d'e-mail client et admin	Vérifier la réception des mails de confirmation	E-mail HTML reçu avec récapitulatif et montant	Conforme
11	Suppression des données temporaires	Vérifier la session après validation	Les données booking_data sont supprimées	Conforme
12	Affichage responsive	Test sur mobile et tablette	Le menu devient burger, mise en page fluide	Conforme

8.2 Résultats et corrections apportées

L'ensemble des tests fonctionnels réalisés sur le site se sont révélés conforme aux attentes.

Le parcours utilisateur est fluide et sans erreur :

- les validations bloquent correctement les cas invalides (dates, e-mails, champs vides),
- le calcul des prix et des acomptes est fiable,
- les e-mails de confirmation et d'administration fonctionnent avec succès,
- l'expérience mobile est satisfaisante.

Les corrections effectuées pendant les tests ont été :

- ajustement de la validation des dates pour éviter les réservations rétroactives,
- correction d'un bug de calcul de prix lors des séjours d'une seule nuit,
- nettoyage automatique des espaces et majuscules dans les champs de formulaire,
- amélioration de la gestion des sessions pour éviter la duplication de réservations.

8.3 Améliorations envisagées

Afin d'accroître la robustesse et la maintenabilité du projet, plusieurs améliorations techniques sont prévues :

- automatiser les tests fonctionnels complets du parcours utilisateur avec Selenium ou Playwright,
- générer automatiquement un PDF récapitulatif de réservation pour le client,
- mettre en place un système de surveillance d'erreurs (ex. : Sentry) en production,
- étendre la couverture des tests unitaires à 100 % du code critique,
- tester le site sur PostgreSQL lors du passage en hébergeur définitif.

9. Veille technologique et sécurité

9.1. Tests unitaires et fonctionnels (Pytest)

Les tests unitaires et fonctionnels ont été réalisés avec Pytest, en utilisant des fixtures Django et le mocking (`unittest.mock.patch`) pour isoler les comportements critiques (calculs, vérifications de capacité, envoi d'e-mails, intégration Stripe, ...).

Chaque test vise à vérifier un aspect clé du back-end : modèles, formulaires et vues.

Exemples :

- Formulaires : validation, nettoyage, et normalisation des champs (`BookingForm`, `BookingDetailsForm`),
- Modèles : vérification du calcul des prix, de l'acompte, de la capacité (`Price`, `SupplementPrice`, `Capacity`),
- Vues : contrôle des redirections, du rendu des templates et de la cohérence des messages utilisateurs.

Ces tests assurent la fiabilité du système et la non-régression lors des mises à jour et une meilleure maintenabilité du code à long terme.

9.2. Scénarios de tests principaux et résultats

Les tests couvrent l'ensemble des fonctionnalités critiques de l'application :

- Les formulaires (`BookingFormClassic`, `BookingDetailsForm`, `ReservationRequestForm`) : vérification des champs obligatoires, validation et nettoyage des données (emails, noms, téléphone) et les cas spécifiques tel que la longueur du câble pour l'électricité ou la taille du véhicule sont couverts,

```
def test_invalid_phone(self):
    """Phone with invalid characters should raise error"""
    form_data = {
        "first_name": "John",
        "last_name": "Doe",
        "address": "123 Test St",
        "postal_code": "33000",
        "city": "Bordeaux",
        "phone": "abc123",
        "email": "john@example.com",
    }
    form = BookingDetailsForm(data=form_data)
    assert not form.is_valid()
    assert "phone" in form.errors
```

- Les modèles (Booking, Price, SupplementPrice, Capacity, MobilHome, SupplementMobilHome, SeasonInfo) : validation des règles métiers, calcul du total et de l'acompte, contrôle de la capacité pour éviter l'overbooking, création et traduction des saisons et des mobil-homes, représentation textuelle des objets,

```
@pytest.mark.django_db
def test_supplementmobilehome_creation():
    """Test SupplementMobileHome object creation and string representation."""
    smh = SupplementMobileHome.objects.create(
        mobile_home_deposit=400,
        cleaning_deposit=80,
        bed_linen_rental=20
    )
    assert str(smh) == "Suppléments mobil-home"
    assert smh.mobile_home_deposit == Decimal('400')
```

- Les vues et URL : vérification des redirections, du rendu des templates, des messages utilisateurs et du bon déclenchement des processus internes (paiement, e-mails, sessions, ...),

```
@patch("bookings.models.Booking.calculate_total_price", return_value=Decimal("100.00"))
@patch("bookings.models.Booking.calculate_deposit", return_value=Decimal("30.00"))
def test_booking_summary_displays_correct_prices(mock_deposit, mock_total, client, valid_booking_data):
    """Booking summary view should correctly display total, deposit, and remaining balance."""
    set_booking_session(client, valid_booking_data)
    url = reverse("booking_summary")
    response = client.get(url)

    assert response.status_code == 200
    assert response.context["total_price"] == Decimal("100.00")
    assert response.context["deposit"] == Decimal("30.00")
    assert response.context["remaining_balance"] == Decimal("70.00")
```

- Les objets de contenu du site (horaires, services, tarifs) : contrôle de la cohérence des informations affichées et de leurs traductions multilingues.

9.3. Difficultés rencontrées et solutions apportées

Difficulté	Description	Solution mise en œuvre
Gestion des dates invalides	Les tests échouaient pour des dates identiques ou inversées	Ajout d'une validation stricte dans BookingForm.clean()
Données de session incomplètes	Les tests de réservation échouaient en absence de booking_data	Vérification et initialisation automatique de la session
Données e-mails non traduites	Certains e-mails restaient en français	Application dynamique de gettext et switch_language()
Interaction avec check_capacity	Nécessité d'isoler le comportement	Utilisation du mock @patch("bookings.models.Booking.check_capacity")

```
def test_missing_required_field(self, mock_check_capacity):
    """Missing conditional field (cable_length) should raise error"""
    today = timezone.localdate()
    form_data = {
        "booking_type": "tent",
        "start_date": today + timedelta(days=1),
        "end_date": today + timedelta(days=3),
        "adults": 2,
        "children_over_8": 1,
        "children_under_8": 0,
        "pets": 0,
        "electricity": "yes",
        "tent_width": 3,
        "tent_length": 4,
    }
    form = BookingFormClassic(data=form_data)
    assert not form.is_valid()
    assert "cable_length" in form.errors
```

L'ensemble des tests, manuels et automatisés, confirme la stabilité, la sécurité et la cohérence du back-end Django.

Les pratiques de test avec Pytest et la validation continue du code garantissent une base solide pour les futures évolutions du site.

10. Analyse des difficultés et solutions

10.1. Organisation et priorisation

Le développement a suivi une approche incrémentale avec des sprints courts et des branches Git dédiées pour chaque fonctionnalité.

Les principales priorités ont été définies selon :

- les fonctionnalités critiques : réservation en ligne, calcul des tarifs, envoi d'e-mails.
- l'expérience utilisateur : affichage responsive, navigation multilingue, validation des formulaires.
- la sécurité et la fiabilité : gestion des données sensibles, protection CSRF, nettoyage des sessions.

Cette organisation a permis de traiter d'abord les fonctionnalités à fort impact métier, puis d'ajouter les améliorations et correctifs.

10.2. Problèmes techniques front-end et back-end

Les problèmes techniques liés au front-end ont été :

- l'adaptation du design responsive sur tous les supports, notamment le rendu des formulaires et des grilles tarifaires sur mobile,
- la gestion des textes multilingues dans les templates et e-mails.

Les problèmes techniques liés au back-end ont été :

- la vérification de la disponibilité et le calcul automatique des prix pour différents types d'emplacements.
- la validation stricte des formulaires pour éviter les erreurs ou données incohérentes,

- l'isolation des fonctions critiques lors des tests unitaires (mocking) pour garantir la fiabilité,
- la gestion des sessions pour conserver temporairement les données de réservation sans duplication ni perte.

Les problèmes techniques transversaux ont été :

- la synchronisation front/back pour que les champs conditionnels (ex. longueur de câble pour l'électricité) soient cohérents entre affichage et logique métier,
- le maintien d'un suivi clair via Git pour éviter les conflits entre branches lors de l'ajout de fonctionnalités simultanées.

10.3. Solutions mises en œuvre et apprentissages

Pour assurer la fiabilité et la cohérence du projet, une attention particulière a été portée aux tests et à la validation. Des tests unitaires, fonctionnels et manuels ont été mis en place afin de détecter rapidement les erreurs et d'y apporter des corrections efficaces. Le recours au mocking a permis d'isoler les composants critiques, garantissant ainsi la précision des résultats sans interférences externes.

Parallèlement, le code a bénéficié d'une amélioration continue grâce à des opérations régulières de refactoring sur les modèles et les vues, ce qui a simplifié la logique métier et facilité la maintenance. La veille technologique et le respect des bonnes pratiques ont été des points clés : l'adoption de Django 5, Bootstrap 5 et Pytest, l'application rigoureuse des conventions de commits Git et la sécurisation des données ont contribué à la robustesse et à la qualité du projet.

Ces pratiques ont également été l'occasion de renforcer les compétences techniques et organisationnelles, en particulier la maîtrise des tests automatisés, la gestion efficace des interactions front-end et back-end, ainsi que l'adaptation des formulaires aux besoins métiers. L'ensemble de ces mesures a permis non seulement de sécuriser le projet, mais aussi d'améliorer la qualité du code et la maintenabilité de l'application, tout en consolidant mon expérience et mon savoir-faire.

11. Conclusion

11.1. Bilan technique et compétences acquises

Le développement du site web du Camping Le Maine Blanc m'a permis de concevoir une architecture Django robuste et modulaire, intégrant un front-end responsive avec Bootstrap 5, un système de réservation dynamique, la gestion de la sécurité des données et des paiements, ainsi qu'une interface multilingue optimisée pour le référencement (SEO).

La mise en place de tests unitaires et fonctionnels avec Pytest, le refactoring régulier et la gestion des branches Git ont renforcé mes compétences techniques et méthodologiques.

11.2. Clarté, pertinence et prise de recul sur le projet

Ce projet a favorisé le développement de compétences transversales essentielles comme :

- autonomie dans la recherche de solutions techniques,
- exploitation de la documentation officielle et de ressources spécialisées,
- rigueur dans l'écriture et la maintenance du code.

La gestion efficace des interactions front/back, la validation des formulaires et la couverture complète des tests m'ont permis d'assurer la cohérence, la fiabilité et la maintenabilité du site.

11.3. Perspectives d'évolution ou améliorations futures

Plusieurs axes d'amélioration ont été identifiés pour les évolutions futures :

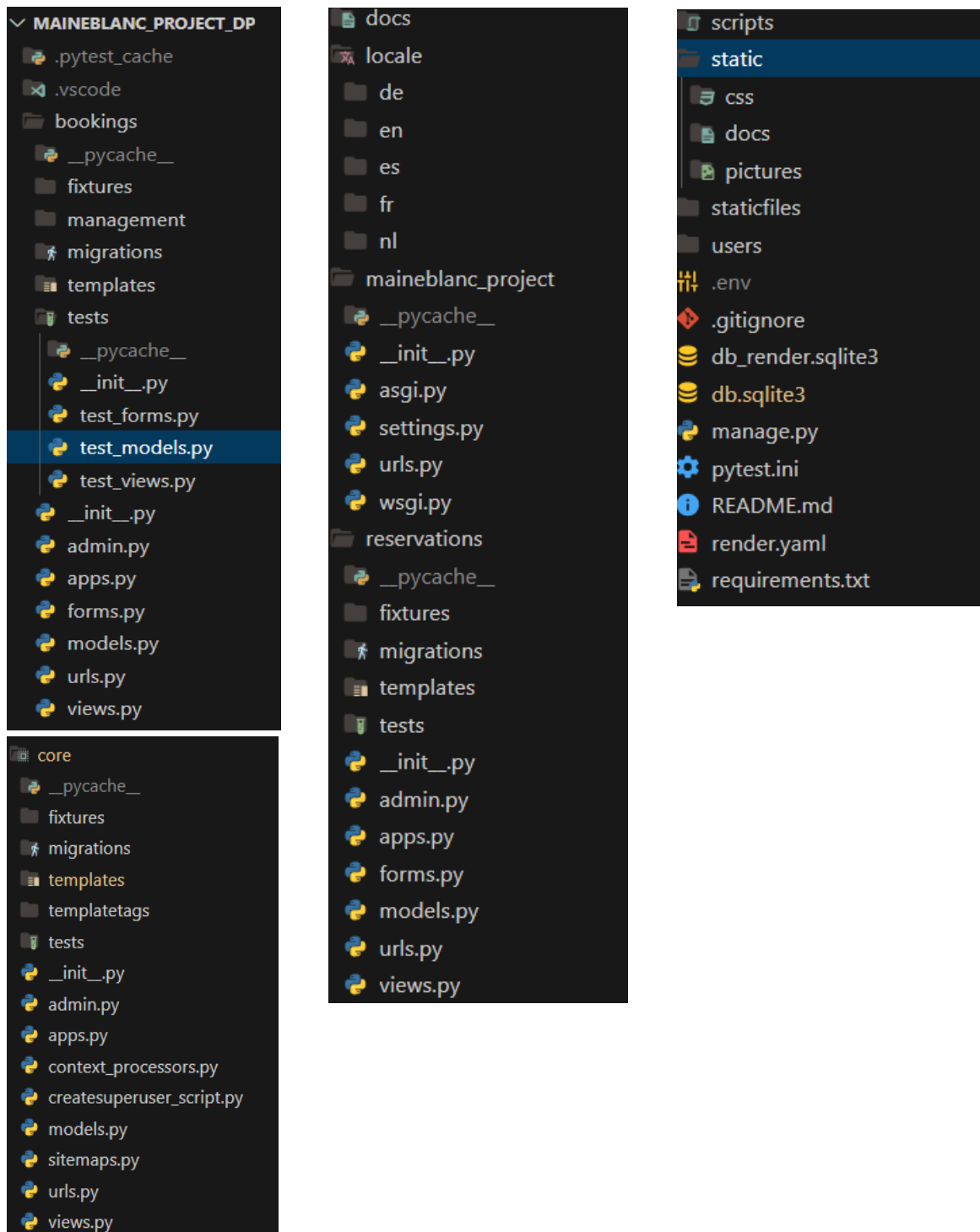
- automatisation complète des tests fonctionnels avec Selenium ou Playwright,
- génération de récapitulatifs PDF pour les clients,
- passage à PostgreSQL sur un hébergement définitif,

- surveillance en temps réel des erreurs et optimisation continue des performances.

Ces perspectives permettent de garantir l'évolution technique et la qualité du site sur le long terme.

12. Annexes

12.1. Arborescence du projet Django

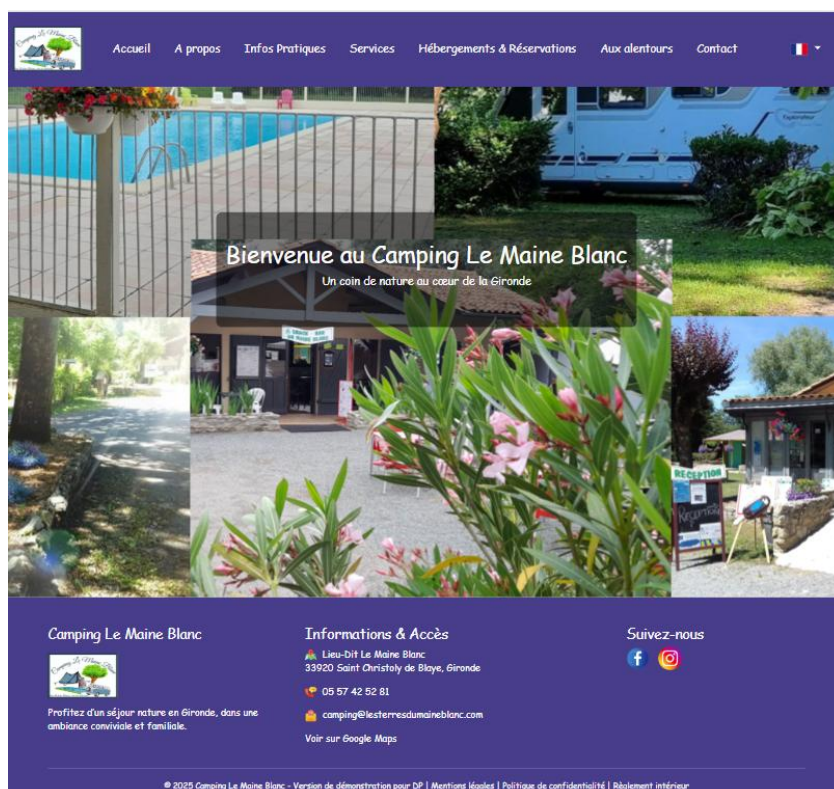


12.2. Technologies utilisées

Type	Outil / Technologie	Utilisation principale
Langage	Python 3 / JavaScript	Back-end (Django) / Interactivité
Framework	Django 5 / Bootstrap 5	Structure web et responsive design
Base de données	SQLite	Environnement de développement
Serveur de test	Django runserver	Exécution locale
Versioning	Git / GitHub	Suivi des versions et sauvegarde du projet
Tests	Pytest / Django TestCase	Vérification du fonctionnement des modèles
Traduction	Django Parler / gettext	Gestion multilingue (FR, EN, ES, DE, NL)
Outils de développement	VS Code	IDE principal
Déploiement (prévu)	Gunicorn + Nginx sauf si autre choix des gérants pour l'hébergeur	Hébergement futur en production

12.3. Exemples d'interfaces

Page d'accueil :



Formulaire de réservation :

[Accueil](#) [A propos](#) [Infos Pratiques](#) [Services](#) [Hébergements & Réservations](#) [Aux alentours](#) [Contact](#) 

Réserver un emplacement

Pour les réservations ouvriers, merci de contacter directement le camping au 05 57 42 52 81 ou via notre formulaire de contact.

Emplacements *
Type d'emplacement
--- Choisissez ---

Dates du séjour *
Date d'arrivée :
Date de départ :

Nombre d'occupants *
Adultes : Enfants +8 ans : Enfants -8 ans : Animaux (2 max) :

Electricité *
☒ Avec électricité ☐ Sans électricité

Réserver

* Champ obligatoires

**Camping Le Maine Blanc**

Informations & Accès
 Lieu-Dit Le Maine Blanc
33920 Saint Christoly de Blaye, Gironde
 05 57 42 52 81

Suivez-nous
 

Récapitulatif de réservation :

Réservation confirmée

**Dates :**
31/10/2025 — 01/11/2025

VOS COORDONNÉES
Nom : Doe
Prénom : John
Adresse mail : johndoe@test.com
Téléphone : 0606060606

NOMBRE DE PERSONNES
Adultes : 2
Enfants + 8 ans : 0
Enfants - 8 ans : 1
Animaux : 1

EMPLACEMENT / VÉHICULE
Type d'emplacement : Caravane
Longueur du véhicule : 6,00 m
Électricité : Avec électricité
Câble électrique : 2,5 m

Total à payer : 24,40 €
Acompte versé : 3,66 €
Solde restant à payer le jour de l'arrivée : 20,74 €

Faire une nouvelle réservation

Interface d'administration Django :

The screenshot shows the Django administration interface. At the top, a blue header bar contains the title 'Administration de Django' and a navigation menu with links: 'BIENVENUE, DEMO_ADMIN_DP', 'VOIR LE SITE', 'MODIFIER LE MOT DE PASSE', and 'DÉCONNEXION' with a sun icon. Below the header, the page is titled 'Site d'administration'. The main content area is divided into two columns. The left column contains three sections: 'AUTHENTIFICATION ET AUTORISATION' with links for 'Groupes' and 'Utilisateurs' (each with '+ Ajouter' and 'Modification' icons); 'INFORMATIONS DIVERSES' with links for 'Laverie', 'Modalités du camping', 'Piscine', and 'Restauration' (each with '+ Ajouter' and 'Modification' icons); and 'PRIX ET RÉSERVATIONS' with links for 'Capacités d'emplacements', 'Dates des saisons', 'Mobil-homes', 'Prix des Suppléments', 'Réservations', 'Suppléments mobil-home', 'Tarifs', and 'Tarifs et infos divers' (each with '+ Ajouter' and 'Modification' icons). The right column contains a box titled 'Actions récentes' with a sub-section 'Mes actions' showing 'Aucun(e) disponible'.

12.4. Liens utiles

- Documentation officielle Django : <https://docs.djangoproject.com/>
- Documentation Bootstrap 5 : <https://getbootstrap.com/docs/5.3/>
- OWASP Cheat Sheet - Django Security : <https://cheatsheetseries.owasp.org/>
- GitHub du projet : https://github.com/Nathi33/maineblanc_project_DP.git
- Maquettage complet : https://github.com/Nathi33/maineblanc_project/releases/tag/v1.0-maquettage
- Version de démonstration déployée sur Render : <https://maineblanc-project-dp.onrender.com>
- Administration du site :
 - <https://maineblanc-project-dp.onrender.com/admin/>
 - Identifiants de connexion (démonstration) :
 - Nom d'utilisateur : Demo_admin_DP
 - Adresse mail : admin_dp@test.com
 - Mot de passe : Demo1234 !

12.5. Glossaire

- Back-end : partie serveur d'un site web gérant la logique et la base de données
- Front-end : partie visible du site (interface utilisateur)
- Responsive Design : Adaptation automatique de l'affichage selon la taille de l'écran
- Framework : ensemble d'outils facilitant le développement d'applications
- ORM (Object Relational Mapper) : outil permettant d'interagir avec la base de données via des objets Python
- Test unitaire : test automatisé qui vérifie le bon fonctionnement d'une partie du code
- Session : données temporaires stockées côté serveur pendant la navigation